# 교사 개인정보 저장 위치 · 암호화 가이드

작성일 2026-04-09 · Git 푸시 전에 반드시 읽어야 하는 문서

> **핵심 한 줄: 회원가입 데이터는 Vercel이 아니라 "Vercel이 연결한 Postgres DB"에 저장되고, 비밀번호는 bcrypt, 이름·이메일·전화는 AES-256-GCM으로 암호화되어 저장된다.**

## 1. 저장 위치 — 환경별

### 1-1. 로컬 개발 (지금 테스트 중)

- 파일 기반 **SQLite**: `backend-node/db.sqlite`
- 본인 맥북 안에만 존재
- Git에 포함되지 않도록 이미 관리됨

### 1-2. Git 푸시 후 Vercel 자동 배포 (운영)

- 환경변수 `POSTGRES_URL` 이 가리키는 외부 Postgres에 저장
- Vercel 자체는 "앱 실행 환경"이고, 데이터는 **별도 DB 서비스**에 쌓인다
- 가능한 제공자: **Vercel Postgres / Neon / Supabase** 중 하나

**연결된 DB 제공자 확인 방법**

```
Vercel 대시보드
  → onlyteaching 프로젝트
  → Settings
  → Environment Variables
  → POSTGRES_URL 값의 호스트 확인

예시:
  postgres://...@*.neon.tech/...              → Neon
  postgres://...@*.supabase.co/...             → Supabase
  postgres://...@*.vercel-storage.com/...      → Vercel Postgres
```

> 💡 **NestJS 앱 코드 (app.module.ts):** `NODE_ENV === 'production'` 이고 `POSTGRES_URL` 이 있으면 Postgres, 아니면 SQLite. 이 분기 한 줄이 로컬/운영 저장소를 나눈다.

## 2. users 테이블 — 컬럼별 저장 형태

| 컬럼 | 저장 형태 | 복호화 가능? | DB 유출 시 노출? |
|---|---|---|---|
| `schoolCode` (아이디) | 평문 | - | ⚠️ 노출 |
| `passwordHash` | bcrypt 해시 | ❌ 불가능 | 안전 |
| `nameEnc` | AES-256-GCM | 키 있어야 가능 | 안전 |
| `emailEnc` | AES-256-GCM | 키 있어야 가능 | 안전 |
| `emailHash` | SHA-256 (중복 체크용) | ❌ 불가능 | 안전 |
| `phoneEnc` | AES-256-GCM | 키 있어야 가능 | 안전 |
| `schoolName` | 평문 (공개 정보) | - | 노출 (의도적) |
| `status` | 평문 ('PENDING' / 'ACTIVE') | - | 노출 (민감 X) |

> ✅ **결론:** DB 덤프가 통째로 유출돼도 공격자가 얻는 건 **아이디 · 학교명 · 가입 상태**뿐. 비밀번호/이름/이메일/전화번호는 못 읽는다.

## 3. 비밀번호 저장 방식 — bcrypt 해시

### 3-1. 원리

- bcrypt는 **단방향 함수**. 해시를 보고 원본 비밀번호를 역산할 수 없음.
- 로그인 시에는 사용자가 입력한 비밀번호를 똑같이 bcrypt 돌려서 **해시끼리 비교**.
- Salt(랜덤값)가 섞여서 같은 비밀번호도 저장된 해시가 매번 다르다.
- Work Factor(cost=10) 때문에 계산이 일부러 느리다 → 무작위 대입 공격 차단.

### 3-2. 실제 저장 예시

```
원본: password123
저장: $2b$10$K8dZgQX...(60자 해시)

같은 비밀번호를 두 계정이 써도 저장된 해시는 다르다:
A계정: $2b$10$K8dZgQX...
B계정: $2b$10$H2fJyLP...
```

### 3-3. 깨는 데 걸리는 시간 (일반 영문숫자 8자)

| 방식 | 소요 시간 |
|---|---|
| MD5/SHA-256 (잘못된 방식) | 1초 이내 |
| **bcrypt cost=10 (우리 방식)** | **약 100년** |

## 4. 이름·이메일·전화 저장 방식 — AES-256-GCM

### 4-1. 원리

- **대칭 암호화**: 잠글 때(암호화)와 열 때(복호화)에 같은 열쇠(key) 사용.
- 열쇠는 DB에 두지 않고 **Vercel 환경변수**(DB 바깥)에 둔다.
- DB가 털려도 열쇠가 없으면 암호문은 그냥 알 수 없는 문자열.
- 매 암호화마다 랜덤 IV(초기화벡터)가 섞여서 같은 평문도 다른 결과가 나온다.
- 변조 감지용 Auth Tag까지 붙어서 누가 암호문을 조작하면 복호화 단계에서 실패한다.

### 4-2. 실제 저장 예시

```
원본 이름: 이재홍
암호문(base64): A2xkL9q8zY1pVvWu/3JhN8Qc5tPRFyr...

복호화 시에는:
  저장된 암호문 + ENCRYPTION_KEY → 원본 "이재홍" 복구
```

### 4-3. 검색 필요한 필드 — 해시 컬럼 병행

> 이메일은 "중복 체크"가 필요한데 암호문으로는 검색이 불가능하다.
> → 별도 `emailHash` 컬럼에 **SHA-256(이메일)**을 저장.
> → 중복 체크는 해시 비교로, 복호화는 암호문으로 처리.

## 5. ⚠️ 운영 배포 전 반드시 할 일

> **가장 중요**: 현재 코드는 `ENCRYPTION_KEY` 환경변수가 없으면 **개발용 폴백 키**로 동작한다. 이 상태로 운영 배포하면 AES 암호화가 사실상 무의미해진다. 반드시 아래 단계를 먼저 수행해야 한다.

### Step 1. 암호화 키 생성 (64자 hex)

```
터미널에서 실행:
node -e "console.log(require('crypto').randomBytes(32).toString('hex'))"

출력 예시:
a3f9b2c1d4e5f67890abcdef1234567890abcdef1234567890abcdef12345678

→ 이 64자 문자열을 복사해둔다. 절대 git에 올리지 말 것.
```

### Step 2. Vercel 대시보드에 환경변수 등록

1. https://vercel.com 접속
2. onlyteaching 프로젝트 클릭
3. 상단 메뉴 **Settings**
4. 왼쪽 메뉴 **Environment Variables**
5. **Add New** 버튼 클릭
6. 입력:
   Key: `ENCRYPTION_KEY`
   Value: (Step 1에서 복사한 64자)
   Environment: Production, Preview, Development 전부 체크
7. **Save** 클릭
8. Deployments 탭에서 최신 배포 우측 ⋯ → **Redeploy**

### Step 3. 키 안전하게 백업

이 키를 잃어버리면 **모든 암호화된 이름·이메일·전화를 영원히 복호화할 수 없다**. 꼭 안전한 곳에 백업:

- 1Password · Bitwarden 같은 비밀번호 관리자
- 본인만 접근 가능한 클라우드 노트 (Notion private 등)
- 물리 종이 출력해서 금고

여러 곳에 **2중 이상** 백업 권장.

### Step 4. .env 파일은 절대 Git에 올리지 말 것

```
.gitignore 확인:
  .env
  .env.local
  .env.production
위 항목들이 반드시 포함되어 있어야 한다.
```

## 6. 해킹 대응 요약

| 시나리오 | 현재 방어 수준 |
|---|---|
| DB 덤프 유출 | **안전** — 비밀번호 bcrypt, 개인정보 AES, 키는 DB 바깥 |
| SQL Injection | **안전** — TypeORM + 파라미터 바인딩 사용 |
| XSS | **안전** — React 기본 이스케이프 |
| 무작위 로그인 시도 | ⚠️ **미비** — 레이트 리밋 아직 없음 (P1 숙제) |
| 환경변수 유출 | Vercel Secrets로 보호 중. 키 로테이션(6개월)은 별도 운영 숙제 |
| 세션 탈취 | HTTPS + localStorage 기반 (토큰 HttpOnly 쿠키 전환은 추후 과제) |

## 7. 지금 Git 푸시 전 최종 체크리스트

| | 항목 |
|---|---|
| ☐ | Vercel 대시보드에 `ENCRYPTION_KEY` 등록 완료 |
| ☐ | 같은 키 비밀번호 관리자에 백업 |
| ☐ | `.env` / `db.sqlite` 가 `.gitignore` 에 포함 |
| ☐ | 로컬에서 회원가입 → 로그인 → 교사 인증 테스트 완료 |
| ☐ | 기본 테스트 계정( `master0109@naver.com` ) 외에도 신규 가입 흐름 확인 |
| ☐ | 배포 직후 운영 Vercel에 실제 접속해서 로그인 되는지 확인 |

> **한 줄 결론**
> Git 푸시 전에 Vercel에 `ENCRYPTION_KEY` 만 제대로 등록하면, 교사 회원가입 개인정보는 **DB가 털려도 읽히지 않는 수준**으로 보호된다.
> 키 관리만 잘하면 된다.